VI. Evaluation

We evaluate the implemented datapath at the boundary that the current evidence supports: cycle-accurate RTL execution with abstract DMA interfaces, coupled to a packet-level network model. Five questions guide the evaluation. Where does the TX pipeline spend time? When does packet spraying help? How much path skew can the receiver absorb? What does ordered completion cost? Finally, what implementation cost has been established, and what remains to be measured on a target FPGA? The results expose both benefits and limits. In the tested single-QP workload, packet spraying raises network goodput by up to $2.48\times$; with four active QPs and 4-KiB messages, however, spraying delivers 26.4% lower goodput than fixed placement. The trace combines byte imbalance with a different packet-emission schedule, so the experiment does not isolate their individual contributions. Receive-window pressure and global retirement are separate constraints that additional path bandwidth alone does not remove.

A. Methodology and Evidence Boundary

Archived co-simulation. The measured batch connects a Verilator simulation of the TX RTL, an ns-3 network, and a Verilator simulation of the RX RTL through Unix sockets [1], [2]. TX exports each frame with the simulated time at which its final beat leaves the packet scheduler. ns-3 injects the frame at that time, models serialization, queuing, and propagation, and forwards the arrival event to RX. Timestamped advance messages coordinate the three processes. Thus, the network may reorder frames without replacing the production times chosen by the RTL. Table I summarizes the archived configuration.

TABLE I
ARCHIVED RTL/NETWORK CO-SIMULATION CONFIGURATION. LINK RATES AND THE 2-NS RTL CYCLE ARE MODEL PARAMETERS, NOT MEASUREMENTS OF A PLACED FPGA.

Component	Configuration
Toolchain	BSC 2023.01; Verilator 5.048; ns-3 3.48; cocotb 2.1.0.dev0
RTL	2-ns model cycle; 4 QPs; 4 receive lanes; messages up to 16 KiB
Receive state	16-packet window per QP; 16 payload slots per lane; 4 Ordered Queue entries per QP
Network	2 nodes; 4 independent 100-Gb/s point-to-point links; 100-packet queue per link
Path delay	200 ns on all paths for rate/policy comparisons; controlled skew for receive-pressure experiments
DMA model	TX completions returned in reverse batches of up to 8 requests; RX completion acknowledged after payload collection
Packet profile	Private Ethernet/IPv4/UDP/BTH/RETH; ICRC and FCS omitted

In this profile, PMTU is a packetization budget rather than pure payload size: the first packet can carry PMTU-28 B and a continuation packet PMTU-12 B; Ethernet/IPv4/UDP framing is outside this budget. For example, a 4-KiB message at a 4-KiB PMTU becomes 4,068 B plus a 28-B tail. The model exercises request processing, abstract DMA, packetization, path selection, receive-window handling, placement, and ordered retirement. It does not include a physical PCIe/GPU path, a switched congestion topology, ICRC/FCS, or an ACK-to-TX retransmission loop. The four equal-rate links isolate path selection; they do not evaluate SRv6 encapsulation. This scope is narrower than deployment-scale MRC studies [3].

Metrics. TX latency begins when RTL accepts a request and ends at either the first frame's start of frame (SOF) or the latest end of frame (EOF) among all packets of that message. TX effective payload rate divides batch payload bits by the interval from the first accepted request to the last emitted frame. Network goodput instead spans first frame injection to final frame arrival. Per-message network completion spans that message's first injection to its final arrival, excluding request processing before injection and RX processing after arrival. Because these metrics use different boundaries, their numeric rates must not be subtracted as "network overhead."

All plotted points are deterministic finite batches without a separate warm-up interval; they are not steady-state NIC throughput. The eight-message policy comparison supports paired means, not tail-latency estimates. Coordination was checked with 100-ns, 1-μs, and 5-μs advance windows, which required 12, 3, and 2 rounds but produced identical injection times, paths, and arrivals. Free-clock execution and offline replay produced the same events, and 20 per-path packet captures matched reference bytes and timestamps.

The one short-suite skip in Table II is the M6 long-only reorder case; it runs in the long suite. The active repository has since evolved into M0–M8 TX/RX units plus the M9–M11 software-coordinated retransmission seam. The latter is not exercised by this batch. Moreover, the current tree preserves the RTL and cocotb suites but not the external ns-3 adapter, raw event archive, or result parser used here. We therefore treat the figures as archived evidence rather than claim push-button reproduction from the current checkout.

TABLE II
FUNCTIONAL CHECKS ASSOCIATED WITH THE ARCHIVED EXPERIMENT BATCH. RX SHORT AND LONG SUITES OVERLAP AND ARE NOT ADDITIVE COVERAGE.

Check	Exercised behavior	Result
TX regression	Request, DMA, packets, scheduler	64 pass
RX short	Window, placement, retirement, Flag	37 pass; 1 skip
RX long	Extended receive/completion	38 pass
Online reference	6 messages, 4 QPs, unequal paths	11/11 frames; 0 drops
Timing consistency	3 windows, free-clock, replay	Exact match
Reverse DMA	Complete 3,2,1,0 across QPs	Retire 0,1,2,3
Queue capacity	1, 2, 4 Flags; fifth blocked	Overflow caught

B. GPU-Ready to On-NIC Payload: A Code-Derived Cycle Budget

The packet-level results above begin after the NIC has accepted a request. This subsection isolates the preceding control path because it determines how soon the first payload read can start. We statically count the serialized RTL stages in M0 and M1, then express the external GPU, PCIe, and DMA terms symbolically. We do not claim a measured GPU or target-FPGA latency.

Model boundary. The active M0 input is `DecodedGdrpSubmission`; the GPU-side release that makes payload visible, credit reservation, the 8-B Doorbell store, PCIe transport, and address-to-QPN decoding are outside the current RTL. The cycle count below therefore starts when that decoded submission is visible at the M0 input FIFO and ends when the first `DmaReadRequest` is accepted at the M1 interface. The endpoint for payload arrival is the M1 `payloadOut` handshake; compaction, packetization, and M3 are excluded. A production front end also needs address/QPN extraction and Doorbell field decode before this point; we denote that unmodelled term D_{decode} and do not assign it a cycle count.

Why the M0 path is serialized. Each table query supplies the next query's key or permission. The Slot lookup returns the local LKEY, remote base, and length limit; the MR lookup validates that LKEY and returns the PGT offset; the QP reservation returns PMTU and PSN; the PGT lookup translates the first physical page. A burst splitter then emits a 4-KiB-safe, at-most-1-KiB DMA request. In the functional model each table is a one-cycle register lookup. At a target clock, a BRAM-backed PGT may add cycles; the counts in Table III are therefore a lower bound for this RTL organization, not a post-route timing result.

TABLE III
CODE-DERIVED SERIALIZED CYCLE BUDGET. A MODULE-BOUNDARY PIPE TRANSFER COSTS ONE CYCLE BECAUSE `toPipeIn`/`toPipeOut` WRAP NON-BYPASS `mkSizedFIFOF` AND `mkConnection` IS A SEPARATE RULE. COUNTS ASSUME NO STALLS AND ONE-CYCLE TABLE LOOKUPS.

Serialized segment	Rules	Pipe hops	Cycles
M0 Slot query	3	2	5
Frontend–processor handoff	0	1	1
M0 MR query	3	2	5
M0 QP reserve/join	2	2	4
Parsed–PGT issue	1	1	2
M0 PGT query	2	2	4
Page fragment and burst split	1	1	2
M0 subtotal	12	11	23
M0–adapter handoff	0	1	1
Adapter message start	1	0	1
Adapter–synchronizer handoff	0	1	1
Synchronizer message start	1	0	1
Synchronizer–M1 handoff	0	1	1
M1 normalizer, scheduler, reorder install	3	2	5
Total to external DMA request	17	16	33

M0 itself needs 23 cycles: 12 rule firings and 11 module-boundary Pipe hops. The Slot and MR queries each take five cycles (issue, request hop, table rule, response hop, join); the QP reserve/join takes four; the PGT path takes six (four for the query and two for the parsed-to-PGT issue); and the burst splitter adds two. The adapter, synchronizer, and M1 installation add ten cycles, five of which are again Pipe hops. This count describes the `TxMvp` $\rightarrow$ `M0MessageAdapter` $\rightarrow$ `DmaPhaseAIntegration` topology; the standalone Phase-A testbench starts at its own `contextIn`/`chunkIn` and is shorter. Thus

$$C_{\mathrm{req}} = 23 + 10 = 33. \tag{1}$$

from submission visibility to the first external DMA-request handshake. At 250 MHz this is 132 ns; at 500 MHz it is 66 ns. Both are clock conversions of the same RTL path, not measured operating frequencies. The fact that 16 of 33 cycles are Pipe

hops is an artifact of the modular functional MVP; a flattened production integration could remove them, but we have not implemented or measured that variant.

First payload beat. The M1 completion reorder waits until the entire first chunk is received before it drains any beat: `recvComplete` becomes true only after the final 128-B completion beat. If R is the request-to-first-completion delay in cycles and K is the number of 128-B completion beats in the first DMA chunk, the first payload beat and the full chunk appear at

$$C_{\text{first}} = C_{\text{req}} + R + K + 3, \tag{2}$$

$$C_{\text{chunk}} = C_{\text{req}} + R + 2K + 2. \tag{3}$$

These expressions assume one completion beat per cycle, no downstream backpressure, and R expressed in the same NIC clock; if the completion interface uses another clock, convert R to time before adding it. For a 4-KiB-aligned 1-KiB chunk, $K = 8$, so the first beat is $R + 44$ and the complete chunk is $R + 51$; an unaligned chunk has $K = 9$ and adds one cycle to the first beat and two to the complete chunk. The $K + 3$ term is a deliberate consequence of whole-chunk reordering, not a PCIe limit. A streaming reorder policy could expose the first beat earlier, but would no longer guarantee that the chunk's beats leave in order.

Comparison with IBGDA. A conventional GPU-initiated RDMA path first writes a WQE and doorbell record, then rings the NIC doorbell; the NIC fetches the WQE before it can issue the payload DMA. IBGDA and GDAKI move the queue and doorbell into GPU-accessible memory, but retain WQE construction and NIC WQE fetch [4], [5]. A recent measurement of minimal GPU-initiated transports reports that WQE construction, doorbell-record update, and the doorbell store alone cost 0.70 μs in *mini-gda*, and 5.31 μs in the NVSHMEM public path on an H200/CX-7 200-Gb/s InfiniBand platform [5]. These are GPU-side issue times, not NIC-internal cycles.

TABLE IV
PUBLISHED P50 ISSUE AND PUT+COMPLETION TIMES FOR ONE 8-B OPERATION ON H200/CX-7 200-GB/S IB [5]. KSM-CYCLE COLUMNS ARE OUR CONVERSION AT AN ASSUMED 2.0-GHZ SM CLOCK, NOT REPORTED BY THE AUTHORS.

Path	Issue (μs)	Put+comp. (μs)	Issue (kSM cyc.)	Put+comp. (kSM cyc.)
mini-gda, inline	0.70	4.03	1.4	8.1
GDAKI, inline	1.60	6.02	3.2	12.1
NVSHMEM, internal	4.26	9.18	8.5	18.4
NVSHMEM, public	5.31	11.10	10.6	22.2

The measured 0.70-μs issue floor is 1.4k SM cycles at 2 GHz. The OpenRDMA M0 path is 23 *NIC* cycles (92 ns at 250 MHz, 46 ns at 500 MHz); the full path to an external DMA request is 33 cycles (132/66 ns). These quantities are in different clock domains and must not be subtracted as cycles. Their comparison is meaningful only in time: the on-NIC control path is small relative to the measured GPU issue floor, so the GPU fence, MMIO write, and PCIe transport—not the table lookups or Pipe hops—are the likely dominant terms.

The architectural difference is better seen as a transaction count. For one 1-KiB operation, Table V lists the control and data movement. IBGDA must publish a WQE and a doorbell record, and the NIC must fetch the WQE before it can fetch the payload. OpenRDMA publishes only the 8-B Doorbell; its Slot, MR, QP, and PGT state lives on the NIC, so there is no WQE fetch. The NIC-side replacement is the 33-cycle M0/adapter/M1 control path (23 cycles within M0).

TABLE V
SUBMISSION-PATH TRANSACTIONS FOR ONE 1-KIB OPERATION. WQE SIZES ARE AT LEAST ONE 64-B BASIC BLOCK [5]; PAYLOAD IS FETCHED BY THE NIC IN BOTH DESIGNS.

Path	GPU-side publication	NIC-side dependent reads
OpenRDMA	8-B Doorbell	payload only
IBGDA	WQE + dbrec + 8-B Doorbell	WQE, then payload

Assuming the same GPU release, Doorbell write, PCIe transport, and payload read, let T_{IBGDA} and T_{OpenRDMA} denote data-ready to first on-NIC payload; the mechanism difference is

$$T_{\text{IBGDA}} - T_{\text{OpenRDMA}} \approx T_{\text{WQE build}} + T_{\text{dbrec}} + T_{\text{WQE fetch}} - \frac{33}{f_{\text{NIC}}}. \tag{4}$$

The equation isolates the WQE work that OpenRDMA removes and the local table path it adds. It omits IBGDA's unpublished NIC-side WQE-decode cycles, so it is a partial accounting rather than a complete latency model. It does not predict an end-to-end speedup: the common GPU fence, MMIO, and PCIe terms are absent from the active RTL and are not measured

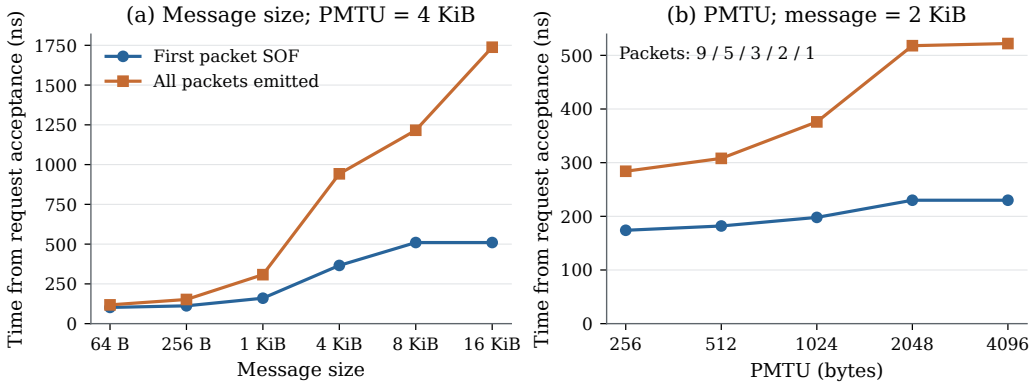


Fig. 1. Isolated TX timing from request acceptance in deterministic runs. Panel (a) uses a 4-KiB PMTU; panel (b) uses a 2-KiB message. "All packets emitted" uses the latest EOF, including when a small tail finishes before an earlier large packet.

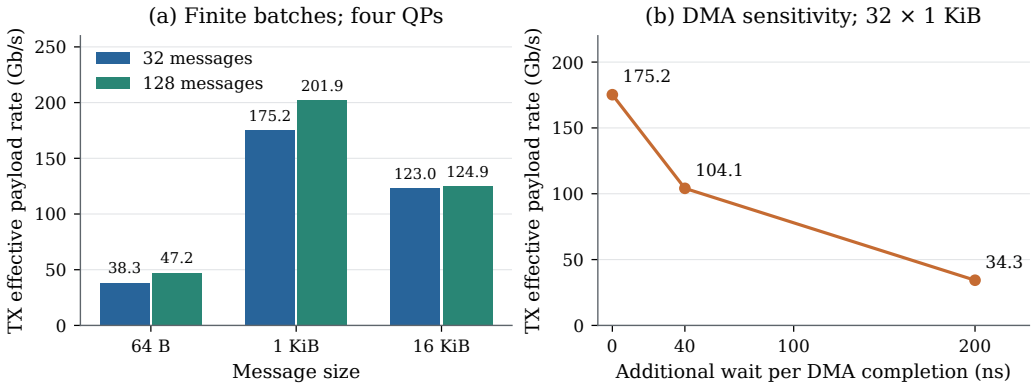


Fig. 2. Finite-batch TX effective payload rate with four active QPs and 200-ns propagation on each of four paths. (a) Deterministic 32- and 128-message batches amortize startup and drain differently. (b) Added wait before each abstract DMA completion lowers the rate of a 32-message, 1-KiB workload.

here. A same-platform experiment must measure those terms for both designs; comparing the 0.70-μs GPU issue floor with NIC RTL cycles directly would be invalid.

The model also explains why the M0 cycle count should not be optimized in isolation. At 250 MHz the complete request path is 132 ns, while the measured minimal GPU issue alone is 700 ns. The useful architectural claim is therefore narrow: OpenRDMA removes WQE construction and a dependent WQE fetch from the NIC's critical path, and replaces them with a 33-cycle on-NIC control path. It does not establish a lower GPU-to-NIC latency without a matched GPU/PCIe/fence measurement.

C. RQ1: Where Does the TX Pipeline Spend Time?

Figure 1 separates startup from complete message emission. At a 4-KiB PMTU, a 64-B request starts its first packet after 102 ns and finishes after 118 ns. For 16 KiB, these boundaries are 510 ns and 1,738 ns. The first-packet delay is also 510 ns at 8 KiB, while latest EOF continues to increase with message size; startup and drain are therefore distinct costs.

Segmentation makes the completion boundary consequential. In the 4-KiB case, the 28-B tail reaches EOF at 380 ns, before the 4,068-B first fragment; the message leaves TX only at 942 ns. Treating the highest packet index as message completion would underestimate emission by 562 ns. For a fixed 2-KiB message, reducing PMTU from 4,096 B to 256 B changes the packet count from one to nine and reduces latest EOF from 522 ns to 284 ns in this model. This result reflects parallel output lanes; it does not establish a network-optimal PMTU because it does not price all real wire and receiver costs.

Figure 2 next exposes startup/drain and DMA dependence. For 1-KiB messages, increasing a four-QP batch from 32 to 128 requests raises TX effective payload rate from 175.2 to 201.9 Gb/s. The corresponding network goodputs are 177.9 and 202.8 Gb/s, but their start boundaries differ. At 64 B, TX reaches 38.3 and 47.2 Gb/s; at 16 KiB, 123.0 and 124.9 Gb/s. Thus, larger messages do not automatically maximize rate in the current pipeline/model.

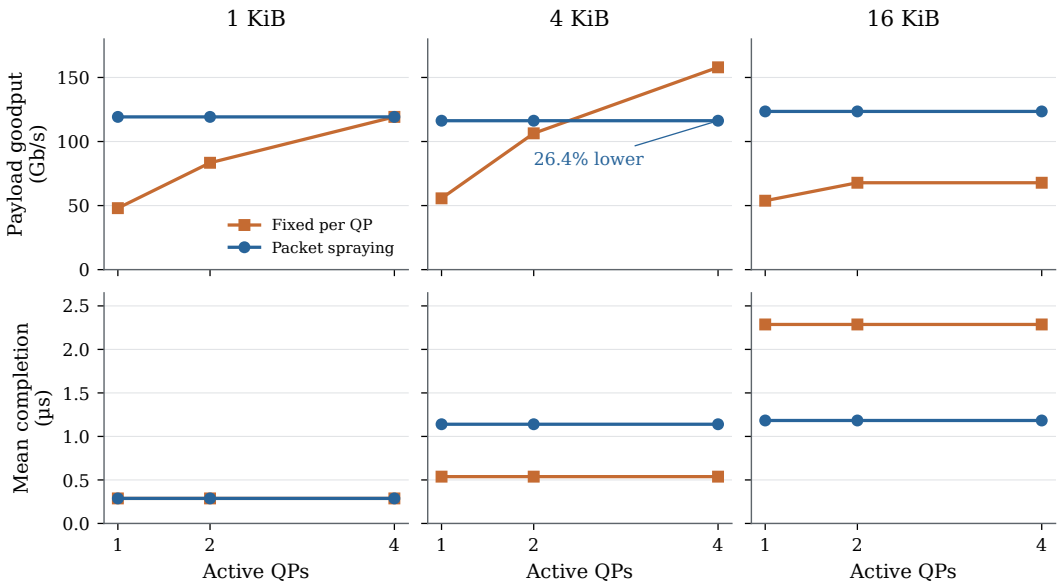


Fig. 3. Packet spraying versus one fixed path per QP for deterministic eight-message batches at a 4-KiB PMTU on four 200-ns paths. The upper row reports batch network goodput; the lower row reports mean first-injection-to-last-arrival completion.

Adding 40 or 200 ns (20 or 100 model cycles) before each DMA completion lowers the 1-KiB, 32-message rate from 175.2 Gb/s to 104.1 or 34.3 Gb/s. Four egress paths cannot compensate for payload that has not reached packetization. These traces establish a DMA-availability dependency, but not its attribution to a specific RTL queue or physical PCIe transaction.

D. RQ2: When Does Packet Spraying Help?

We compare implemented round-robin spraying among ready paths with fixed per-QP placement using the low two QPN bits. Each pair has the same offered sequence of eight messages, size, QP assignment, 4-KiB PMTU, and four 200-ns paths. Policy-dependent backpressure may still change request acceptance and frame injection. Fixed placement is therefore an implementation-level path-selection baseline, not a comparison with a commercial RNIC.

With one active QP, fixed placement leaves three paths idle. Spraying raises 1-KiB goodput from 48.0 to 119.2 Gb/s ($2.48\times$) and 16-KiB goodput from 53.8 to 123.5 Gb/s ($2.30\times$). For 16 KiB, mean network completion falls from 2.287 to 1.183 μs. A 1-KiB message is one packet whose 287.68-ns injection-to-arrival latency does not change; its gain comes from placing successive messages of one QP on different links, not from accelerating a packet.

As active QPs increase, fixed placement occupies more links. At four QPs, both 1-KiB policies reach 119.2 Gb/s. For 4-KiB messages the result reverses: fixed placement reaches 157.9 Gb/s, while spraying reaches 116.2 Gb/s, 26.4% lower; mean completion is 0.538 versus 1.140 μs. Each message consists of a 4,068-B fragment and a 28-B tail. Spraying assigns 12,204, 8,192, 8,164, and 4,208 payload bytes to the four paths, whereas fixed placement assigns 8,192 B to each. Packet-opportunity balance is not byte balance, and the small tail also changes the emission schedule. The experiment does not isolate these two causes, so the supported conclusion is conditional: spraying helps when QP count underutilizes paths, but packet-granular round robin is not uniformly better near segmentation boundaries.

E. RQ3: How Much Path Skew Can RX Absorb?

A 1-KiB message at a 1-KiB PMTU produces 996-B and 28-B packets. We hold three paths at 200 ns and add 0–2,000 ns to the fourth. In a two-message, one-QP burst, mean network completion is 414.44 ns at 0 and 200 ns of extra delay, then 560.08, 810.08, and 1,310.08 ns at 500, 1,000, and 2,000 ns. Initially the delayed tail is not the message's final arrival; once it is, completion follows that path's additional delay.

Longer bursts expose a separate receiver limit. At equal path delay, 11 messages pass and 12 fail; with 1,000 or 2,000 ns of extra delay, nine pass and ten fail. At 200 and 500 ns the sampled pairs are 13/14 and 12/13. Failure is `RX_OUT_OF_WINDOW`, not network loss: every frame arrives in the recorded failing runs. Equal propagation also does not guarantee in-order RTL input because small tails and packets on different lanes have different serialization histories. The non-monotonic 200-ns point is a property of this schedule, not evidence that delay increases capacity. Without a closed retransmission loop, a rejected packet cannot be recovered in this batch.

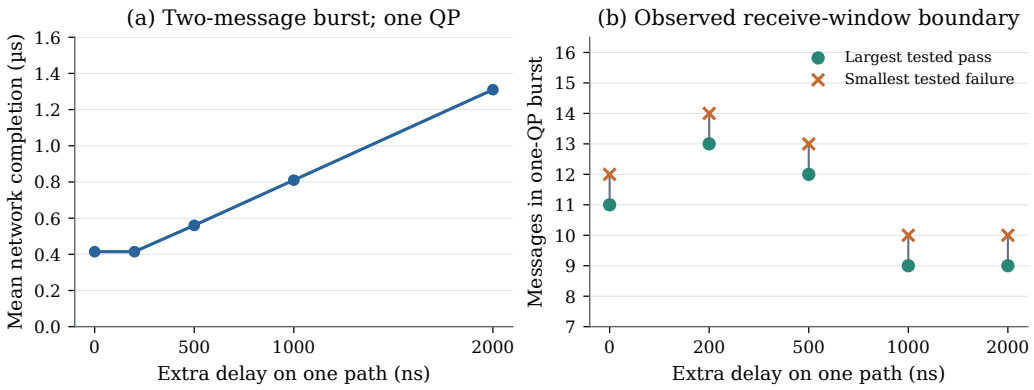


Fig. 4. Receive behavior for deterministic 1-KiB messages at a 1-KiB PMTU, with three 200-ns paths and one slower path. (a) Mean network completion for a two-message, one-QP burst. (b) Sampled largest passing and smallest failing one-QP bursts; markers are not measured occupancy or a general capacity bound.

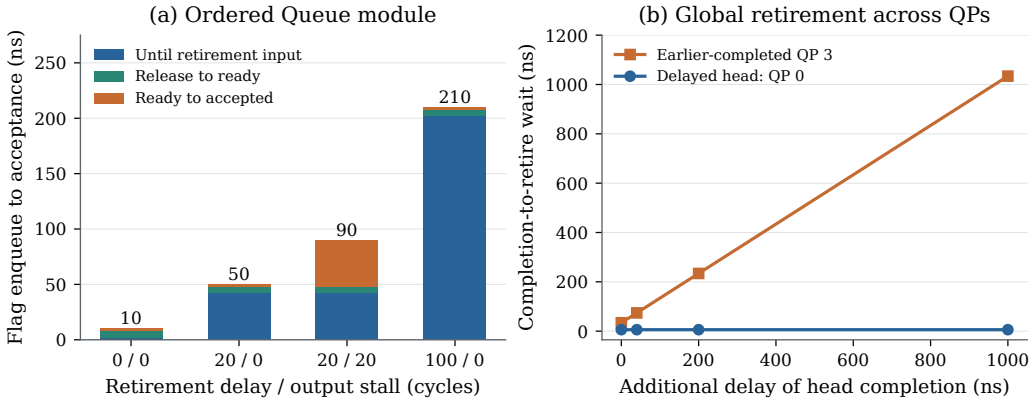


Fig. 5. Controlled ordering tests. (a) Flag release with delayed retirement and output backpressure; one model cycle is 2 ns. (b) Four QPs complete in order 3,2,1,0 but retire in order 0,1,2,3; delaying QP 0 transfers its delay to already-completed QP 3.

F. RQ4: What Does Ordered Completion Cost?

We isolate the Ordered Queue because packet arrival alone does not authorize an Ordered Flag. The testbench enqueues a Flag, injects retirement evidence at a controlled barrier, and optionally stalls the output. The retirement event is an RTL input in this experiment, not a measured GPU-memory DMA completion.

With no output stall, retirement delays of 0, 20, and 100 cycles produce observed Flag enqueue-to-acceptance times of 10, 50, and 210 ns. Combining 20 cycles of retirement delay with 20 cycles of output stall yields 90 ns. In the 100-cycle case, retirement evidence is accepted at 202 ns, the request becomes ready at 208 ns, and it is accepted at 210 ns; credit is dequeued at 212 ns and the modeled response at 214 ns. The two-nanosecond gaps include sequential testbench drive/drain cadence and are not intrinsic interface-latency claims. Capacity tests return one, two, and four credits for the corresponding released Flags and detect a fifth insertion into a blocked four-entry queue.

The second experiment places one request from each of four QPs on different DMA lanes and completes them in reverse order. Retirement remains 0,1,2,3. QP 3, which completes first, waits 34 ns in the baseline and 1,034 ns when QP 0 is delayed by 1,000 ns. QP 0's own completion-to-retire interval remains 6 ns. Thus a delayed global head transfers essentially all of its delay to unrelated, already-completed QPs. Per-QP retirement may remove this cross-QP HoL, but its state and arbitration cost have not been evaluated.

G. RQ5: Established Cost and Remaining Evidence

Table VI groups the declared entry data in the archived RTL inventory. The listed structures contain 2,586,432 logical bits (315.73 KiB). RX payload-frame storage contributes 260 KiB; the four per-QP Ordered Queues together hold only 416 B of

TABLE VI
LOGICAL STATE FOOTPRINT OF LISTED STRUCTURES. COUNTS ARE DECLARED ENTRY DATA, NOT PLACED FPGA MEMORY OR A COMPLETE-TREE RESOURCE ESTIMATE.

Structure	Logical bits	KiB
RX payload frame storage	2,129,920	260.00
TX channel payload storage	262,144	32.00
TX DMA completion reorder	158,848	19.39
RX pending-packet table	13,888	1.70
Per-QP Ordered Queues	3,328	0.41
Other listed state	18,304	2.23
Total listed capacity	2,586,432	315.73

TABLE VII
EVIDENCE CLOSURE FOR AN FCCM IMPLEMENTATION EVALUATION. THE RIGHT COLUMN STATES REQUIRED MEASUREMENTS, NOT INFERRED RESULTS.

Dimension	Evidence here	Required closure
Function	Archived co-simulation and controlled RTL tests	Restore trace-level cross-model artifact
Cycle cost	Endpoint intervals under abstract DMA	Stage, stall, and occupancy counters
Scale/robustness	Finite batches and sampled skew boundary	Warm-up-separated, multi-seed QP/loss sweeps
Resources	315.73 KiB listed logical state	Successful Fitter/TimeQuest and post-route report
Recovery	M9–M11 functional seam outside this batch	Connect M3, M6, runtime, and retry egress
Physical system	Not measured	PCIe/MAC/GPU throughput, latency, and power

entry data. Within this inventory, payload buffering rather than Flag metadata dominates. Increasing Flag queue space would not address receive-window pressure or global-retirement blocking.

Logical bits are not FPGA resource utilization: synthesis may repack memories and add FIFOs, control, routing, and vendor-IP cost. We therefore keep this inventory separate from target-device implementation evidence, as is necessary in FPGA system evaluations [6]. A historical standalone M1 synthesis artifact is excluded because its Fitter failed and it has no placed timing result. Credible FCCM implementation evidence still requires a controlled top and SDC, successful synthesis/Fitter/TimeQuest, module and total ALM/LUT/FF/BRAM/DSP usage, target clock, WNS/TNS, and post-route Fmax. Target-clock feasibility must not be inferred from the 2-ns model cycle.

The present evidence establishes functional datapath execution, conditional multipath gains, schedule-specific sampled receive-window pass/fail boundaries, cross-QP ordering cost, and a logical-state baseline. It does not establish physical-NIC throughput, GPU-visible completion latency, closed-loop loss recovery, SRv6 cost, target-FPGA frequency, measured power, or application speedup. Completing the evaluation requires (i) event-level closure among the reference model, RTL, and restored network harness; (ii) explicit pipeline latency, occupancy, and stall counters; (iii) warm-up/steady-state/drain-separated QP, window, skew, and loss sweeps; and (iv) successful target-device place-and-route followed by physical PCIe/MAC/GPU measurements. These additions strengthen the evidence level without changing the negative results and scope boundaries reported above.

REFERENCES

[1] W. Snyder, "Verilator: Open-Source SystemVerilog Simulator and Lint System," Software, 2003, accessed October 26, 2026. [Online]. Available: https://www.veripool.org/verilator/

[2] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. B. Kopena, "Network Simulations with the ns-3 Simulator," in *ACM SIGCOMM 2008 Conference Demonstration*, Seattle, WA, USA, 2008, p. 527. [Online]. Available: https://conferences.sigcomm.org/sigcomm/2008/papers/p527-hendersonA.pdf

[3] J. Araujo, A. Chow, M. Handley, R. Lewis, C. Paasch, J. Padhye, M. Papamichael, G. Steinbrecher, A. Tootoonchian, L. Yuan *et al.*, "Resilient AI Supercomputer Networking using MRC and SRv6," May 2026, arXiv:2605.04333. [Online]. Available: https://arxiv.org/abs/2605.04333

[4] NVIDIA Corporation, *NVSHMEM Documentation: Using GPUDirect Async Kernel-Initiated Communication*, NVIDIA Corporation, 2026, nVSHMEM 3.7.0; documentation for IBGDA and GPUNetIO GDAKI transports; accessed October 26, 2026. [Online]. Available: https://docs.nvidia.com/nvshmem/api/using.html

[5] J. Baydamirli, I. Ismayilov, K. Oktay, and D. Unat, "GPU-Initiated Communication: Dissecting Down to the Bone," 2026, arXiv:2610.01380. [Online]. Available: https://arxiv.org/abs/2610.01380

[6] C. Wang, K. He, R. Fan, X. Wang, W. Wang, and Q. Hao, "CXL over Ethernet: A Novel FPGA-Based Memory Disaggregation Design in Data Centers," in *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2023, pp. 75–82.